Module 6 Week A — Reading: Text Processing & Named Entity Recognition

Everything in this reading is fair game for readiness checks, drills, and labs. A learner who reads this carefully should be able to complete the Week A drill and lab without attending the concept session.

What success looks like: After this reading, you can explain why text data requires different processing than tabular data, apply text preprocessing techniques (tokenization, normalization, lemmatization), handle multilingual text including Unicode and Arabic considerations, build spaCy Natural Language Processing (NLP) pipelines for part-of-speech (POS) tagging, dependency parsing, and sentence segmentation, implement Named Entity Recognition using both spaCy and Hugging Face, and evaluate NER output quality with precision, recall, and F1.

In This Reading

1. From Tables to Text
2. Text Preprocessing
3. Tokenization in Depth
4. Multilingual Text Handling
5. The spaCy NLP Pipeline
6. Part-of-Speech Tagging and Dependency Parsing
7. Sentence Segmentation
8. Named Entity Recognition
9. Evaluating NER Output
10. The Hugging Face Ecosystem

- Key Terms
- Summary
- What's Next

---

1. From Tables to Text

In Modules 1 through 5, every dataset you worked with was **tabular** – rows of observations, columns of typed features, each cell holding a number or a category. You could count rows, compute means, plot distributions, and feed features directly into scikit-learn. Text is fundamentally different.

Consider this sentence: "Apple is looking at buying U.K. startup for $1 billion." A human reads it and immediately extracts facts: a company (Apple), an action (acquisition), a location (U.K.), and an amount ($1 billion). A computer sees a sequence of characters with no inherent structure. There are no columns, no types, no fixed length.

Text data is:

- **Unstructured** – No schema, no columns. Each document is a variable-length sequence of characters.
- **Variable-length** – A tweet is 280 characters; a legal brief is 100 pages. Unlike tabular data, there is no fixed feature dimension.
- **Ambiguous** – "Bank" means a financial institution or a river's edge depending on context. "Dr." is an abbreviation, a title, and a sentence-ending period depending on what follows. "US" (country) and "us" (pronoun) differ only by case.
- **Context-dependent** – The meaning of a word changes based on surrounding words. "Jordan" is a country, a person's name, or a river depending on the sentence.
- **Multilingual** – In professional contexts in Jordan, the same document may contain English and Arabic text. Each language has different tokenization rules, character sets, and preprocessing needs.

The **Natural Language Processing (NLP) pipeline** is the bridge from raw text to structured data that machines can process. Every technique in this module exists to resolve one or more of these challenges – converting unstructured, ambiguous text into structured, computable representations.

Why does this matter practically? Imagine you are building a system that monitors climate policy documents for mentions of specific organizations and locations. You cannot use `df.groupby("organization")` because there is no "organization" column – the organization names are buried in paragraphs of natural language. NER (Named Entity Recognition) is the technique that extracts those names automatically.

---

2. Text Preprocessing

Before any NLP analysis, raw text needs cleaning. The standard preprocessing pipeline transforms messy, inconsistent text into a normalized form that downstream tools can process reliably.

The Standard Preprocessing Pipeline

The typical sequence is:

1. **Unicode normalization** – Ensure consistent character encoding (Section 4 covers this in depth).
2. **Tokenization** – Split text into individual units (words, subwords, or characters). Section 3 covers tokenization in detail.
3. **Lowercasing** – Convert all text to lowercase so "Climate" and "climate" are treated as the same word.
4. **Punctuation removal** – Strip punctuation marks that do not carry meaning for the analysis.
5. **Stop word removal** – Remove extremely common words ("the", "is", "at", "and") that carry little semantic value.
6. **Lemmatization** – Reduce words to their dictionary form: "running" becomes "run", "policies" becomes "policy", "better" becomes "good".

When to Apply Each Step – and When Not To

Not every step is appropriate for every task:

- **Lowercasing** is standard for most tasks but can hurt NER. "US" (country) becomes indistinguishable from "us" (pronoun). "Apple" (company) becomes "apple" (fruit). If your task relies on capitalization as a signal, delay lowercasing until after NER.
- **Stop word removal** is useful for topic modeling and keyword extraction but can break NER. "The United Nations" becomes "United Nations" – the spans shift, and multi-word entity detection may fail. "New York" might lose "New" if it appears in the stop list.
- **Stemming vs. lemmatization** – Stemming uses crude rules to chop words to approximate roots: "running" becomes "run" (correct), but "university" becomes "univers" (wrong). Lemmatization uses linguistic rules and a dictionary to find the actual root form: "better" becomes "good", "mice" becomes "mouse". **Lemmatization is almost always preferred** because it produces real words. spaCy provides lemmatization via the `token.lemma_` attribute.
- **Punctuation removal** should preserve punctuation that carries meaning. Hyphens in "state-of-the-art" connect a compound adjective. Periods in "U.K." are part of the abbreviation.

A Basic Preprocessing Function

```python
import re

def preprocess_text(text, stop_words=None):
    """Preprocess raw text: lowercase, remove punctuation, tokenize, remove stop
    text = text.lower()
    text = re.sub(r'[^\w\s]', '', text)  # Remove punctuation
    tokens = text.split()
    if stop_words:
        tokens = [t for t in tokens if t not in stop_words]
    return tokens
```

This is a starting point. For more sophisticated preprocessing – especially lemmatization – you will use spaCy (Section 5).

---

3. Tokenization in Depth

Tokenization is the process of splitting text into discrete units called **tokens**. It is the first and most fundamental step in any NLP pipeline. The choice of tokenization strategy has cascading effects on every downstream task.

Word-Level Tokenization

The simplest approach: split on whitespace and punctuation. "The IPCC released its report." becomes `["The", "IPCC", "released", "its", "report", "."]`.

Word-level tokenization is intuitive and works well for many tasks, but it has limitations:

- **Vocabulary explosion** – Every unique word form is a separate token. "run", "runs", "running", "ran" are four different tokens. With a large corpus, the vocabulary can grow to hundreds of thousands of entries.
- **Out-of-vocabulary (OOV) problem** – A word the model has never seen during training cannot be processed. Domain-specific terms like "COP28" or "IPCC-AR6" may not appear in general-purpose vocabularies.
- **Morphologically rich languages** – Languages like Arabic, Finnish, or Turkish can express in a single word what English expresses in a phrase. Word-level tokenization for these languages produces enormous vocabularies.

spaCy uses a word-level, rule-based tokenizer that handles punctuation, abbreviations, and special cases well.

Subword Tokenization

Modern NLP models (BERT, GPT, T5) use **subword tokenization** – they split words into smaller pieces when the full word is not in the vocabulary. Two common algorithms:

- **BPE (Byte Pair Encoding)** – Starts with individual characters and iteratively merges the most frequent adjacent pairs. "tokenization" might become `["token", "ization"]`. Used by GPT-2 and RoBERTa.
- **WordPiece** – Similar to BPE but uses a likelihood-based criterion for merging. "tokenization" might become `["token", "##ization"]` where `##` indicates a continuation piece. Used by BERT.

Why subword tokenization matters:

- **No OOV problem** – Any word can be decomposed into known subword pieces, even words the model has never seen.
- **Compact vocabulary** – A vocabulary of 30,000 subword pieces can represent far more words than 30,000 whole words.
- **Handles morphology** – "unhappiness" might become `["un", "##happi", "##ness"]`, capturing meaningful prefixes and suffixes.

When you use Hugging Face models later in this module, the tokenizer is built into the model – you do not need to choose it manually.

Character-Level Tokenization

The finest granularity: every character is a token. Rarely used alone in modern NLP because it produces very long sequences and loses word-level meaning. However, character-level features are sometimes combined with word-level features for tasks like spelling correction or language identification.

---

4. Multilingual Text Handling

In professional settings in Jordan, bilingual English/Arabic text is the norm. Climate policy documents may contain English technical terms embedded in Arabic text, or Arabic organization names in English reports. Handling multilingual text correctly requires understanding how computers represent characters.

Unicode and UTF-8

Every character on every computer is represented by a number called a **code point**. The **Unicode** standard assigns code points to over 140,000 characters from virtually every writing system. Unicode is the "what" – it defines which number maps to which character.

UTF-8 is the most common **encoding** – it defines how those code points are stored as bytes. English characters use 1 byte each. Arabic characters use 2 bytes. Some emoji use 4 bytes. UTF-8 is the default for Python 3 strings and the web.

Why Encoding Errors Happen

When you read a file saved in one encoding using a different encoding, characters are misinterpreted. A file saved as UTF-8 but read as Latin-1 turns "é" into "Ã©". A file saved as Windows-1256 (Arabic) but read as UTF-8 produces errors or garbled text.

```python
# Reading a file with explicit encoding
with open("article.txt", "r", encoding="utf-8") as f:
    text = f.read()

# Checking a string's encoding
raw_bytes = text.encode("utf-8")
decoded = raw_bytes.decode("utf-8")
```

Unicode Normalization

The same visual character can be represented by different code points. The letter "é" can be a single code point (U+00E9, precomposed) or two code points (U+0065 "e" + U+0301 combining accent). This causes string comparison failures: two strings that look identical are not equal.

```python
import unicodedata

text = unicodedata.normalize("NFC", text)  # Normalize to composed form
```

Always normalize text before processing. **NFC** (Normalization Form C, Canonical Composition) is the standard choice. The algorithm first decomposes characters canonically, then recomposes them into precomposed forms — but the acronym names the form, not the algorithm.

Arabic Text Considerations

Arabic text presents specific challenges:

- **Right-to-left (RTL) display** – Arabic reads right to left. In Jupyter notebooks and terminals, mixed Arabic/English text may display in unexpected order. This is a display issue, not a data issue – the underlying string is stored correctly.
- **Arabic Unicode block** – Arabic characters occupy code points U+0600 to U+06FF. You can detect Arabic text by checking if characters fall in this range.
- **Diacritics (tashkeel)** – Arabic diacritics are optional marks that indicate short vowels. The same word may appear with or without diacritics. Removing diacritics is a common preprocessing step:

```python
import re

def remove_arabic_diacritics(text):
    """Remove Arabic diacritical marks."""
    arabic_diacritics = re.compile(r'[\u0610-\u061A\u064B-\u065F\u0670\u06D6-\u06
    return arabic_diacritics.sub('', text)
```

- **Common preprocessing** – Arabic-specific preprocessing may include: diacritic removal, letter normalization (e.g., different forms of alef), elongation removal (stretching characters for emphasis), and number conversion.

Language Detection

When a dataset contains multiple languages, you need to identify which language each text is written in before applying language-specific processing.

Simple heuristic: check for the presence of Arabic Unicode characters.

```python
def detect_language_simple(text):
    """Simple language detection based on character ranges."""
    arabic_chars = sum(1 for c in text if '\u0600' <= c <= '\u06FF')
    return 'ar' if arabic_chars > len(text) * 0.3 else 'en'
```

For more robust detection, the `langdetect` library provides probabilistic language identification. Note that `langdetect` is **non-deterministic by default** – set a seed for reproducibility:

```python
from langdetect import detect
from langdetect import DetectorFactory
DetectorFactory.seed = 0

language = detect("The climate summit produced new commitments.")  # 'en'
```

---

5. The spaCy NLP Pipeline

spaCy is a production-grade NLP library designed for efficiency and ease of use. Unlike NLTK (which is a teaching toolkit with many options), spaCy makes opinionated choices and provides a single, fast pipeline. You will use spaCy throughout Modules 6 through 9.

Installing and Loading a Model

spaCy separates the library from the language models. After installing spaCy, you download a model:

```
pip install spacy
python -m spacy download en_core_web_sm
```

`en_core_web_sm` is a small English model (~12 MB) trained on web text. It includes a tokenizer, POS tagger, dependency parser, NER, and lemmatizer. Larger models (`en_core_web_md`, `en_core_web_lg`, `en_core_web_trf`) are more accurate but require more memory and compute. For this course, `en_core_web_sm` is the standard – it runs comfortably on any laptop.

```python
import spacy

nlp = spacy.load("en_core_web_sm")
```

The `nlp` object is a callable pipeline. When you call it on text, it runs every pipeline component in sequence:

```python
doc = nlp("The IPCC released its sixth assessment report in 2023.")
```

The Doc Object

`nlp(text)` returns a `Doc` object – a container of `Token` objects with all linguistic annotations attached. The `Doc` is the central data structure in spaCy.

```python
# Iterate over tokens
for token in doc:
    print(token.text, token.pos_, token.dep_, token.lemma_)
```

Token Attributes

Each `Token` object carries a rich set of attributes:

Attribute	Type	Example	Description
`token.text`	str	`"IPCC"`	The original text
`token.lemma_`	str	`"release"`	The dictionary form (lemma)
`token.pos_`	str	`"PROPN"`	Coarse-grained POS tag (universal tag set)
`token.tag_`	str	`"NNP"`	Fine-grained POS tag (Penn Treebank)
`token.dep_`	str	`"nsubj"`	Syntactic dependency relation
`token.is_stop`	bool	`False`	Whether the token is a stop word
`token.is_punct`	bool	`False`	Whether the token is punctuation
`token.is_alpha`	bool	`True`	Whether the token consists of alphabetic characters

Span Objects

A **Span** is a slice of the `Doc` – a contiguous sequence of tokens. Named entities (Section 8) are represented as `Span` objects with a `.label_` attribute.

```python
# Access a span by index
span = doc[1:5]  # Tokens 1 through 4
print(span.text)  # "IPCC released its sixth"
```

Pipeline Components

The spaCy pipeline runs components in order: **tokenizer** (always first, rule-based) then **tagger** (POS tags), **parser** (dependency relations), **NER** (named entities), and **lemmatizer**. You can inspect the pipeline:

```python
print(nlp.pipe_names)  # ['tok2vec', 'tagger', 'parser', 'attribute_ruler', 'lem
```

Not every task needs every component. If you only need tokenization and NER, you can disable the parser to speed up processing:

```python
nlp = spacy.load("en_core_web_sm", disable=["parser"])
```

---

6. Part-of-Speech Tagging and Dependency Parsing

Part-of-Speech (POS) Tags

A **POS tag** identifies the grammatical role of a word: noun, verb, adjective, adverb, etc. spaCy uses the **Universal POS tag set** (coarse tags like `NOUN`, `VERB`, `ADJ`) and also provides fine-grained tags from the Penn Treebank set.

```python
doc = nlp("Climate scientists published groundbreaking research.")
for token in doc:
    print(f"{token.text:20} {token.pos_:8} {token.tag_:6}")
```

Output:

```
Climate             NOUN     NN
scientists          NOUN     NNS
published           VERB     VBD
groundbreaking      ADJ      JJ
research            NOUN     NN
.                   PUNCT    .
```

Common POS tags you will encounter:

Tag	Meaning	Examples
`NOUN`	Common noun	climate, report, policy
`PROPN`	Proper noun	Jordan, IPCC, UNEP
`VERB`	Verb	released, analyzed, reported
`ADJ`	Adjective	environmental, significant, global
`ADV`	Adverb	recently, significantly, globally
`ADP`	Preposition	in, on, for, at
`DET`	Determiner	the, a, this, every
`PUNCT`	Punctuation	. , ! ?

Why POS tags matter: they are features for downstream tasks. Filtering by POS (e.g., extracting only nouns and proper nouns) is a common first step in keyword extraction and topic analysis.

Dependency Parsing

Dependency parsing identifies the syntactic structure of a sentence – which words modify or depend on which other words. Each token has a **head** (the word it depends on) and a **dependency label** describing the relationship.

```python
doc = nlp("The IPCC released its assessment report.")
for token in doc:
    print(f"{token.text:15} --{token.dep_:10}--> {token.head.text}")
```

Output:

```
The            --det       --> IPCC
IPCC           --nsubj     --> released
released       --ROOT      --> released
its            --poss      --> report
assessment     --compound  --> report
report         --dobj      --> released
.              --punct     --> released
```

Key dependency labels:

Label	Meaning	Example
`nsubj`	Nominal subject	"IPCC" is the subject of "released"
`dobj`	Direct object	"report" is the object of "released"
`det`	Determiner	"The" modifies "IPCC"
`compound`	Compound modifier	"assessment" modifies "report"
`ROOT`	Root of the sentence	"released" is the main verb

Visualizing Dependencies

spaCy's `displacy` module provides beautiful visualizations:

```python
from spacy import displacy

doc = nlp("Climate policy drives economic transformation.")
displacy.render(doc, style="dep", jupyter=True)
```

This renders an arc diagram showing the dependency structure. In a Jupyter notebook, the visualization appears inline. For scripts, use `displacy.serve(doc, style="dep")` to open a browser visualization.

---

7. Sentence Segmentation

Sentence segmentation (also called sentence boundary detection) identifies where one sentence ends and the next begins. This sounds simple – split on periods, right? In practice, it is surprisingly hard.

Why Sentence Segmentation Is Hard

- "Dr. Smith presented at the U.N. headquarters." – The periods after "Dr" and "U.N" are abbreviations, not sentence boundaries.
- "The temperature rose by 2.5 degrees. The trend was significant." – The period after "2.5" is a decimal, not a sentence boundary.
- Bullet lists, headings, and tables often lack terminal punctuation.
- Informal text (chat, social media) may lack periods entirely.

spaCy's Sentence Boundary Detection

spaCy detects sentence boundaries as part of the dependency parser. Sentences are accessed via `doc.sents`:

```python
doc = nlp("The IPCC released its report. It covers 30 years of climate data. Poli
for sent in doc.sents:
    print(sent.text)
```

Output:

```
The IPCC released its report.
It covers 30 years of climate data.
Policy recommendations follow.
```

spaCy's parser-based approach handles abbreviations, decimal numbers, and most edge cases correctly because it considers the full syntactic context, not just punctuation rules.

When Sentence Segmentation Matters

Sentence segmentation is essential for:

- **Document structure analysis** – How long are the sentences? How complex?
- **NER at the sentence level** – Analyzing which entities co-occur in the same sentence.
- **Summarization** – Selecting key sentences from a document.
- **Translation** – Translating sentence by sentence.

For this module, you will use sentence segmentation to structure your NER analysis – extracting entities per sentence and analyzing co-occurrence patterns.

---

8. Named Entity Recognition

Named Entity Recognition (NER) identifies and classifies mentions of real-world entities in text. It turns unstructured text into structured facts: given "The IPCC released its report in Geneva," NER produces `("IPCC", ORG)` and `("Geneva", GPE)`.

NER is one of the most practical NLP tasks. It is the foundation for information extraction, knowledge graph construction, and structured search.

Standard Entity Types

The standard entity type schema includes:

Entity Type	Description	Examples
`PERSON`	People (names, titles)	Dr. Ahmed, the lead researcher
`ORG`	Organizations	IPCC, UNEP, World Bank
`GPE`	Geopolitical entities (countries, cities, states)	Jordan, Amman, the United States
`LOC`	Non-GPE locations	the Dead Sea, the Arctic, Mount Everest
`DATE`	Dates and periods	2023, last year, the 21st century
`MONEY`	Monetary values	$1 billion, 500 million euros
`CARDINAL`	Numerals not covered by other types	two, 3.5 million, 100
`PERCENT`	Percentages	50%, 2.5 percent

spaCy NER

spaCy's built-in NER model identifies entities as `Span` objects in `doc.ents`:

```python
doc = nlp("The IPCC met in Geneva to discuss the Paris Agreement in 2023.")

for ent in doc.ents:
    print(f"{ent.text:25} {ent.label_:10} ({ent.start_char}, {ent.end_char})")
```

Output:

```
IPCC                      ORG        (4, 8)
Geneva                    GPE        (16, 22)
the Paris Agreement       LAW        (34, 53)
2023                      DATE       (57, 61)
```

Each entity has:

- `ent.text` – The entity text as it appears in the document
- `ent.label_` – The entity type label
- `ent.start_char` / `ent.end_char` – Character offsets in the original text

Visualizing Entities

```python
from spacy import displacy

doc = nlp("Jordan committed $500 million to renewable energy by 2030.")
displacy.render(doc, style="ent", jupyter=True)
```

This highlights entities inline with color-coded labels – a quick way to inspect NER quality.

Hugging Face NER Pipeline

The **Hugging Face (HF)** `transformers` library provides a simple NER pipeline interface ("HF" is the conventional shorthand used throughout this module):

```python
from transformers import pipeline

ner = pipeline("ner", model="dslim/bert-base-NER")
result = ner("The IPCC released a report about climate change in Geneva.")
```

The output is a list of dictionaries:

```
[
    {"entity": "B-ORG", "word": "IP", "start": 4, "end": 6, "score": 0.99},
    {"entity": "I-ORG", "word": "##CC", "start": 6, "end": 8, "score": 0.99},
    {"entity": "B-LOC", "word": "Geneva", "start": 52, "end": 58, "score": 0.98}
]
```

Handling Subword Tokens in Hugging Face NER

Notice two things about the raw output above:

1. **Subword splits.** "IPCC" is split into two pieces: "IP" and "##CC". This is because the BERT tokenizer uses **WordPiece** subword tokenization. The `##` prefix indicates a continuation token – it is part of the previous word.
2. **IOB label prefixes.** The `entity` field reads `B-ORG`, `I-ORG`, `B-LOC` – not just `ORG` or `LOC`. This is the **IOB tagging scheme** (also called BIO): every token carries a position tag indicating whether it is the **B**eginning of an entity, **I**nside an entity span, or **O**utside any entity. Token-classification NER models emit per-token tags because entities can span multiple tokens; the position prefix is what lets downstream code reconstruct the span. spaCy hides this machinery from you – `doc.ents` returns already-merged spans with bare labels – but Hugging Face exposes the raw per-token stream, so you have to collapse it yourself.

To reconstruct full entity text and clean labels, you need to both merge continuation tokens **and** strip the `B-` / `I-` prefix. The helper below does both:

```python
def merge_subword_entities(ner_results):
    """Merge subword tokens into complete entities."""
    merged = []
    current = None

    for token in ner_results:
        if token["entity"].startswith("B-"):
            if current:
                merged.append(current)
            current = {
                "entity": token["entity"][2:],
                "word": token["word"],
                "start": token["start"],
                "end": token["end"],
                "score": token["score"]
            }
        elif token["entity"].startswith("I-") and current:
            # Continuation: merge into current entity
            word_piece = token["word"].replace("##", "")
            current["word"] += word_piece
            current["end"] = token["end"]
            current["score"] = min(current["score"], token["score"])
        else:
            if current:
                merged.append(current)
                current = None

    if current:
        merged.append(current)
    return merged
```

The `token["entity"][2:]` slice is what strips `B-` / `I-` off the label so the output uses bare types (`ORG`, `LOC`). Both steps – WordPiece merging and IOB prefix stripping – are essential whenever you use a Hugging Face token-classification NER model. Without them you get fragmented entity text and labels that don't match spaCy's conventions.

Comparing spaCy and Hugging Face NER

spaCy and Hugging Face NER models often produce different results on the same text:

Aspect	spaCy (`en_core_web_sm`)	HF (`dslim/bert-base-NER`)
Architecture	Transition-based, optimized for speed	Transformer (BERT), optimized for accuracy
Entity types	18 types (PERSON, ORG, GPE, DATE, MONEY, ...)	4 types (PER, ORG, LOC, MISC)
Speed	Very fast	Slower (transformer inference)
Subword handling	Word-level tokens; no merging needed	WordPiece tokens; requires merging
Best for	Production pipelines, large-scale processing	High-accuracy extraction on smaller datasets

Neither is universally better. Choose based on your requirements: speed vs. accuracy, entity type granularity, and domain match.

---

9. Evaluating NER Output

NER models make mistakes. They miss entities, mislabel them, or get the boundaries wrong. Systematic evaluation is essential before deploying any NER system.

Entity-Level Evaluation

NER evaluation is done at the **entity level**, not the token level. An entity is counted as correct only if **both the text span and the label match** the gold standard.

- **True Positive (TP)** – The model predicted an entity, and it matches the gold standard (same text, same label).
- **False Positive (FP)** – The model predicted an entity that does not exist in the gold standard, or predicted the wrong label.
- **False Negative (FN)** – The gold standard has an entity that the model missed entirely.

Examples:

Gold Standard	Model Prediction	Evaluation
("IPCC", ORG)	("IPCC", ORG)	TP – exact match
("United Nations", ORG)	("Nations", ORG)	FP + FN – boundary mismatch
("Jordan", GPE)	("Jordan", PERSON)	FP + FN – label mismatch
("Dead Sea", LOC)	(not found)	FN – missed entity
(nothing)	("2023", DATE)	FP – spurious entity

Precision, Recall, and F1

These metrics summarize NER quality:

- **Precision** = TP / (TP + FP) – Of all entities the model predicted, how many were correct?
- **Recall** = TP / (TP + FN) – Of all entities that exist, how many did the model find?
- **F1** = 2 * (Precision * Recall) / (Precision + Recall) – The harmonic mean, balancing precision and recall.

High precision, low recall: the model is conservative – it finds fewer entities but the ones it finds are usually correct. High recall, low precision: the model is aggressive – it finds most entities but includes many false alarms.

Why Token-Level Metrics Are Misleading for NER

If a text has 100 tokens and only 5 are entity tokens, a model that predicts zero entities achieves 95% token-level accuracy by simply labeling everything as "not an entity." This is useless. Entity-level precision, recall, and F1 ignore non-entity tokens and focus on the entities themselves.

Computing NER Metrics in Practice

```python
def evaluate_ner(predicted_entities, gold_entities):
    """Compute entity-level precision, recall, and F1.

    Args:
        predicted_entities: list of (entity_text, entity_label) tuples
        gold_entities: list of (entity_text, entity_label) tuples

    Returns:
        dict with precision, recall, f1 keys
    """
    pred_set = set(predicted_entities)
    gold_set = set(gold_entities)

    tp = len(pred_set & gold_set)
    fp = len(pred_set - gold_set)
    fn = len(gold_set - pred_set)

    precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
    recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
    f1 = 2 * precision * recall / (precision + recall) if (precision + recall) >

    return {"precision": precision, "recall": recall, "f1": f1}
```

Manual Inspection

Metrics tell you *how well* the model performs, but not *why* it fails. Always inspect a sample of errors:

- Which entity types are most often missed?
- Are boundary errors common (e.g., "United" extracted instead of "United Nations")?
- Does the model struggle with domain-specific entities (e.g., "COP28", "IPCC AR6")?
- Are there systematic patterns in the errors?

This qualitative analysis guides decisions about whether to use custom rules, fine-tune the model, or switch to a different model.

---

10. The Hugging Face Ecosystem

The **Hugging Face** ecosystem is the standard toolkit for working with transformer-based NLP models. You will use it throughout Modules 6 through 9. This section introduces the key components.

The `transformers` Library

The `transformers` library provides:

- **Models** – Pre-trained transformer architectures (BERT, DistilBERT, RoBERTa, GPT-2, T5, etc.)
- **Tokenizers** – Fast tokenizers matched to each model's vocabulary
- **Pipelines** – High-level Application Programming Interface (API) for common tasks (NER, sentiment analysis, summarization, etc.)

The pipeline API is the fastest way to get started:

```python
from transformers import pipeline

# Named Entity Recognition
ner = pipeline("ner", model="dslim/bert-base-NER")
result = ner("Jordan committed to renewable energy targets.")

# Sentiment Analysis
sentiment = pipeline("sentiment-analysis")
result = sentiment("The climate outlook is concerning.")

# Zero-Shot Classification
classifier = pipeline("zero-shot-classification")
result = classifier("Global temperatures rose by 1.5 degrees.",
                    candidate_labels=["climate", "economy", "sports"])
```

The `datasets` Library

The `datasets` library provides access to thousands of NLP datasets through a unified API:

```python
from datasets import load_dataset

# Load a standard NER dataset
dataset = load_dataset("conll2003")
print(dataset["train"][0])
```

For this module, you will work with a provided climate articles dataset rather than a HF dataset. In later modules, `datasets` becomes more central.

The Model Hub

The Hugging Face Model Hub (huggingface.co/models) hosts over 200,000 pre-trained models. You can search by task (NER, text classification, translation), language (English, Arabic, multilingual), and domain. Key models for this module:

- `dslim/bert-base-NER` – English NER, 4 entity types (PER, ORG, LOC, MISC)
- `en_core_web_sm` (spaCy) – English NLP pipeline with 18 entity types
- `Davlan/xlm-roberta-base-wikiann-ner` – Multilingual NER (used in Week A stretch assignments)

How spaCy and Hugging Face Connect

spaCy and Hugging Face serve complementary roles:

- **spaCy** is optimized for production NLP pipelines: fast, memory-efficient, batteries-included (tokenizer + POS + NER + parser in one call). Best for large-scale text processing.
- **Hugging Face** provides access to state-of-the-art transformer models: higher accuracy, broader model selection, active research community. Best for tasks where accuracy matters most.

In this course, you will use spaCy as your primary NLP toolkit and Hugging Face for specific tasks where transformer models add value (NER comparison, embeddings, fine-tuning). Both libraries are essential tools in a modern NLP engineer's toolkit.

Key Terms

- **Tokenization** – The process of splitting text into discrete units (tokens) for processing.
- **Token** – A single unit of text produced by tokenization (a word, subword, or character).
- **Subword tokenization** – A tokenization strategy that splits words into smaller pieces; handles out-of-vocabulary words.
- **BPE (Byte Pair Encoding)** – A subword tokenization algorithm that iteratively merges the most frequent character pairs.
- **WordPiece** – A subword tokenization algorithm used by BERT; continuation tokens are marked with `##`.
- **Normalization** – Converting text to a standard form (lowercasing, Unicode normalization, diacritic removal).
- **Lowercasing** – Converting all text to lowercase to treat different cases as the same word.
- **Stemming** – Reducing words to approximate roots by removing suffixes; produces non-words ("univers" from "university").
- **Lemmatization** – Reducing words to their dictionary form using linguistic rules ("better" becomes "good", "mice" becomes "mouse").
- **Lemma** – The dictionary form of a word.
- **Stop word** – An extremely common word (the, is, at, and) that carries little semantic value; sometimes removed during preprocessing.
- **Unicode** – The standard that assigns a unique code point to every character in every writing system.
- **UTF-8** – The most common encoding for Unicode text; English uses 1 byte per character, Arabic uses 2.
- **Encoding** – The method used to convert characters to bytes for storage.
- **Code point** – The unique number assigned to a character in the Unicode standard.
- **NLP pipeline** – A sequence of processing steps that transforms raw text into structured annotations.
- **spaCy** – A production-grade NLP library for Python with built-in models for tokenization, POS tagging, NER, and more.
- **Doc** – spaCy's container object for a processed text; holds Token and Span objects with annotations.
- **Token (spaCy)** – A spaCy object representing a single token with attributes like `.text`, `.pos_`, `.dep_`, `.lemma_`.
- **Span** – A contiguous slice of a spaCy Doc; entities and sentences are Span objects.
- **Part-of-speech (POS) tag** – A label indicating a word's grammatical role (NOUN, VERB, ADJ, etc.).
- **Universal POS tag set** – A standardized set of coarse POS tags used across languages.
- **Dependency parsing** – Identifying the syntactic structure of a sentence; which words modify or depend on which other words.
- **Dependency tree** – The tree structure produced by dependency parsing; each word has a head and a dependency label.
- **Head** – In a dependency tree, the word that another word depends on.
- **Dependent** – In a dependency tree, a word that modifies or is governed by its head.
- **Sentence segmentation** – Identifying sentence boundaries in text.
- **Named Entity Recognition (NER)** – The task of identifying and classifying mentions of real-world entities in text.
- **Named entity** – A mention of a real-world entity (person, organization, location, date, etc.) in text.
- **Entity label** – The category assigned to a named entity (PERSON, ORG, GPE, DATE, etc.).
- **PERSON** – NER label for people.
- **ORG** – NER label for organizations.
- **GPE** – NER label for geopolitical entities (countries, cities, states).
- **LOC** – NER label for non-GPE locations (mountains, bodies of water, regions).
- **DATE** – NER label for dates and time periods.
- **MONEY** – NER label for monetary values.
- **Precision (entity-level)** – Of all entities the model predicted, the fraction that are correct.
- **Recall (entity-level)** – Of all entities that exist in the gold standard, the fraction the model found.
- **F1 (entity-level)** – The harmonic mean of precision and recall.
- **API (Application Programming Interface)** – The set of functions and classes a library exposes for programmatic use; a "high-level API" wraps complex internals behind a small surface.
- **HF (Hugging Face)** – Conventional shorthand for the Hugging Face library ecosystem (`transformers`, `datasets`, the Model Hub).
- **Hugging Face transformers** – A Python library providing pre-trained transformer models and a pipeline API for NLP tasks.
- **Pipeline (Hugging Face)** – A high-level API that wraps a model and tokenizer for a specific task (NER, classification, etc.).
- **Model Hub** – The Hugging Face repository of pre-trained models, searchable by task, language, and domain.

Summary

This reading covered the complete foundation for text processing and Named Entity Recognition:

1. **From tables to text** – Text is unstructured, variable-length, ambiguous, and context-dependent. The NLP pipeline bridges the gap between raw text and structured data.
2. **Text preprocessing** – The standard pipeline: tokenization, normalization, stop word removal, lemmatization. Each step has tradeoffs – preprocessing must be task-aware.
3. **Tokenization** – Word-level (spaCy), subword (BERT/GPT), and character-level approaches. Subword tokenization solves the out-of-vocabulary problem.
4. **Multilingual handling** – Unicode, UTF-8 encoding, Arabic text considerations (RTL, diacritics, normalization), and language detection.
5. **spaCy pipelines** – The `nlp(text)` call processes text through tokenization, POS tagging, dependency parsing, NER, and lemmatization. Doc, Token, and Span objects carry all annotations.
6. **POS tagging and dependency parsing** – Linguistic annotations that reveal grammatical structure. Used as features for downstream tasks.
7. **Sentence segmentation** – Parser-based boundary detection handles abbreviations and decimals correctly.
8. **Named Entity Recognition** – Extracting structured facts (entity text + entity type) from unstructured text. spaCy and Hugging Face provide complementary NER capabilities.
9. **Evaluating NER** – Entity-level precision, recall, and F1. Token-level metrics are misleading. Manual inspection reveals error patterns.
10. **The Hugging Face ecosystem** – transformers, datasets, and the Model Hub provide access to state-of-the-art NLP models.

What's Next

In **Week B**, you will move from extracting entities (structured output from text) to representing text as numbers. You will learn count-based representations (bag-of-words, TF-IDF), static word embeddings (word2vec concepts, GloVe vectors), and contextual embeddings from BERT/DistilBERT. The culminating deliverable combines this week's NER pipeline with Week B's embeddings into an end-to-end NLP analysis system.

© 2026 LevelUp Economy. All rights reserved. Unauthorized reproduction or distribution of this material is prohibited.